

自编码器实验手册

第058单元 用重构和线性探测评价同一表示

一 准备环境与事先写下判断标准

本实验从原创八维合成数据开始，训练二维基础自编码器AE和去噪自编码器DAE，对照原始输入、PCA和随机网络。实验完成的标志不是loss变小，而是能逐项说明：每种表示如何获得、哪些信息进入了训练、重构与下游分类为什么可能给出不同答案。建议先用纸笔完成讲义的二维例子，再用60至90分钟按本手册操作。

不需要GPU，不下载数据或模型，不调用API。运行前准备Python 3.12，创建独立环境。若使用Anaconda Prompt，在058目录执行：

```
conda env create -f environment.yml
conda activate dl058
python --version
python -c "import torch; print(torch.__version__)"
python -c "import numpy; print(numpy.__version__)"
```

也可使用venv，在激活后执行 `python -m pip install -r requirements.txt`。不要在另一个终端未激活环境时直接运行Notebook，否则import成功与否可能取决于错误解释器。本包记录的实际验证版本为Python3.12.14、PyTorch2.14.1+cpu、NumPy2.3.5、Matplotlib3.10.8；requirements允许兼容的PyTorch2系列，不承诺跨版本逐位一致。

打开experiment.py，找到make_data、Autoencoder、reconstruction_loss、fit_probe和run。暂时不用改代码。把以下协议写进实验记录：训练640例、验证160例、测试320例，表示训练不读标签；探测只读训练前128标签；瓶颈2；步数250；批64；学习率0.005；三种子5801至5803；DAE噪声标准差0.5；探测岭系数1；每run120秒保护上限。

先提出两个可被否定的预测：AE重构会优于未训练网络；DAE对同分布附加噪声的恢复会优于AE。第三个问题不预设答案：更好的重构是否带来更好的线性探测？即使实际结果与预测不同，也保留全部结果，不追加种子直到出现想要的数字。

目录中的outputs是随包参考结果。自己的重放写入replay，避免覆盖参考文件；Notebook使用notebook_replay。两者都可删除后重新生成，不需改动讲义数值。图形依赖Noto Sans CJK字体；若机器没有字体，可先完成数值实验，按README安装字体后再画中文图。

二 用小测试先验证问题定义

先运行两次测试，第二次显式开启Python优化模式。终端应返回PASS，进程退出码为0。测试采用显式异常，不能因为-O删除assert而得到假通过。

```
python test_experiment.py
python -O test_experiment.py
```

如果手算MSE测试失败，先检查是否把两个坐标的平方和误当平均；若编码器梯度失败，检查是否漏掉从两个输出回到z的求和；若PCA误差失败，核对总残差除以“样本数乘维度”。检查点测试失败时，不要随意删掉坏文件绕过，应先确认模型结构和保存数组是否对应。

接着在Python或Notebook逐行检查数据。下面代码只查看本地生成结果，不训练模型：

```
import numpy as np
from experiment import make_data, Autoencoder
D = make_data()
for name in ["train", "validation", "test"]:
    x, y = D["x_"+name], D["y_"+name]
    print(name, x.shape, y.shape, np.unique(y, return_counts=True))
print(sum(p.numel() for p in Autoencoder().parameters()))
```

预期形状依次为(640,8)、(160,8)、(320,8)，对应标签长度640、160、320；每个划分两类各一半；模型参数总数362。训练前128例不保证恰好每类64，它是固定样本前缀，不为平衡标签而重新抽样。全训练均值只从x_train计算，不把validation或test拼进去。

建立一次独立数值核对

手写四个点(3,0)、(-3,0)、(0,1)、(0,-1)，执行np.linalg.svd。取vt[:1].T编码、再乘vt[:1]解码，确认MSE=0.25。你应该能解释：两点在第一轴被精确重构，另两点的第二坐标丢失；残差平方和是2，而平均除数是8。

再把一个探测训练特征固定为常数列，测试特征换成很大的数。fit_probe保存的均值和尺度必须仍由训练特征决定。这个检查模拟一种常见泄漏：代码表面没有用测试标签，却用测试分布提前改善了预处理。转导式设置不是永远不允许，但本课明确定义为未见测试样本的归纳评价，不能悄悄变更。

把手算与测试输出放入自己的报告。只贴一张“运行成功”的截图，无法说明你理解了损失、形状和数据边界。

三 真正训练并追踪每一个输出

在058目录运行：

```
python experiment.py --output replay
python make_figures.py --results replay/results.json --output replay/figures
```

主程序先保存protocol.json和data.npz，再计算PCA及基线，最后训练六个网络。每run固定250步，不自动加长。单次训练如果超过120秒会报错；这时检查CPU负载、环境和线程，而不是把失败结果静默排除。实际用时取决于机器，参考执行在CPU上很快；首次import与启动耗时不等于results.json内记录的内部工作计时。

基础AE和DAE同种子使用相同初始化、样本顺序。每步代码中的clean来自64行训练输入，corrupted为clean加新噪声，损失目标仍为clean。AE的sigma=0，DAE为0.5。标签数组虽然保存在数据字典中，但训练loss不引用它；有标签操作集中在add中的fit_probe调用。

核对保存文件而不是只读终端

- data.npz：三个划分的x与y及固定旋转矩阵；读取时用allow_pickle=False。
- pca.npz：训练输入均值、 8×2 方向矩阵和全部奇异值。
- weights_ae_5801.npz等六份文件：全部模型参数及输入均值。
- probe_*.npz共11份：有标签训练特征均值、尺度、线性权重和320个测试分数。
- plot_data.npz：供画图使用的潜在表示、重构与标签，不是新的训练数据。
- results.json：11行完整结果、六条训练轨迹、PCA理论残差和实际软件版本。

每个训练轨迹应含步1、50、100、150、200、250，名称clean_validation_mse说明评价用干净验证输入。minibatch_loss对DAE使用受扰输入，因此二者不是同一个样本集上的同一个问题。不要拿它们的差直接计算“泛化差距”。

从保存权重恢复

创建Autoencoder，用np.load打开一份权重，将除input_mean外的数组复制为torch张量并load_state_dict。取测试输入减保存均值，在eval和no_grad下推断；所得中心化重构与plot_data对应数组应在数值容差内一致。若要展示原始观测，再把均值加回。测试会对全部六份权重重复这一核对。

训练结束后关闭解释器，再启动新进程重跑。真正独立的重放应无需你在先前终端手动定义任何变量。这是保存代码、环境和数据协议的目的。

四 按三个评价问题整理结果

从replay/results.json读取表格，不手工抄图估数。对random2、AE、DAE分别计算三种子的均值与范围；raw与PCA各一个确定性结果，不伪造误差条。每个accuracy乘320必须对应保存的整数正确数。不得用三个种子的最高值代替方法结果。

问题	输入给模型什么	比较目标	指标
干净重构	中心化干净测试输入	同一个干净测试输入	所有 320×8 坐标MSE
附加噪声恢复	测试输入加固定标准差0.5噪声	干净测试输入	同样的坐标MSE
标签可线性读取吗	干净测试输入的冻结表示	二分类标签	320例准确率

参考结果中，PCA2干净重构0.038963，AE三种子平均0.042547，DAE约0.046456；探测分别约47.81%、47.81%、48.02%。raw重构为0而探测100%，但保留了8维；不能宣称它在同一压缩率下击败2维方案。随机网络的解码器没有训练，因此其大重构误差只代表未训练网络的输出。

去噪指标中，raw约0.243129、PCA约0.096840、AE约0.113834、DAE约0.099736。计算DAE相对AE的绝对改善及相对改善比例；注意“下降0.0141个MSE单位”与“下降约12.4%”不是同一句话。PCA仍略好于DAE，需要保留在结论中。

打开五张结果图，先读轴、图例、样本数，再看形状。训练曲线纵轴对数化，不可用视觉垂直距离直接解释为固定绝对误差差；潜在图坐标可以被旋转或缩放，不能按图中横轴长度比较信息多少。标签颜色混杂可作为提示，但最终依据是明确读出器和测试结果。

Notebook路径

启动jupyter lab experiment.ipynb，选择本环境Python内核，执行Restart Kernel and Run All。Notebook不仅加载参考结果，也会生成notebook_replay，重新训练全部六次，核对数值，展示本次图。随包已保存一次顺序执行结果。受构建环境限制，验证使用新Python进程中的in-process IPython内核；这验证了逐单元格执行，未验证浏览器Jupyter界面或网络传输。

自己的运行如果与参考有小数差异，先核对库版本、CPU/GPU、线程与代码；不改参考答案去迎合结果。若结论方向改变，应保留差异和配置，并检查是否已不再运行同一个协议。

五 独立扩展 交付与常见失败

只改一个明确因素

扩展题选择 $K \in \{1, 2, 3, 4, 8\}$ ，检查标签相关方向能否在更宽瓶颈中保留。另建实验副本，给模型增加显式bottleneck参数，修改对应decoder输入维度，并写出参数公式。固定数据、三种初始化、每run250步、批次顺序、探测标签预算与岭系数。每次运行前先保存新协议。

选择规则可以是“不选最佳K，预先报告所有K的完整曲线”。若要推荐一个K，则先定义只使用验证信息的选择指标；此时需要为探测器额外设计验证标签预算，且明确说明与原实验不同。不要拿已展示的320个测试标签反复挑参数，最后再把最高准确率称作未见泛化。

扩展需要同时报告维度、参数量、干净重构、去噪指标和线性探测。K变大可能降低重构误差，也可能使分类更容易，但不能提前填写“必然100%”。如果K=8模型仍不如恒等映射，应检查训练预算与优化问题，不把可表达的恒等函数误当成已学到的恒等函数。

最小提交内容

1. 一段说明任务、数据与信息边界的文字，列出固定协议及任何偏离。
2. 二维网络损失、梯度和更新的纸笔推导，以及四点PCA的数值核对。
3. 原始11行结果与三种子汇总，不删除不利种子；给出所有正确数分母。
4. 至少三张有标题、轴和样本数的图，并解释每张图不能证明什么。
5. 用保存权重恢复一次推断的结果，附普通模式与-O模式的测试输出。
6. 不超过200字的主结论，注明合成数据、一个固定划分、固定线性探测和维度不等。

排查顺序

loss异常大先查shape、目标和中心化；NaN先查学习率、输入有限性和除0；DAE没有恢复作用先查目标是否仍是干净x；探测异常高先查标签泄漏和是否使用测试统计；重构接近0先查K、跳连与是否真的评价未见输入。Notebook在中途运行成功但Run All失败，通常意味着隐藏状态或路径依赖，应从头修复。

若只缺中文字体，数值测试仍可完成；不要因此改动实验算法。若新机器版本导致数值差异，记录实际版本与差值，并根据数学测试和结果范围判断，不轻率宣称逐位复现。所有扩展到此为止，不自动下载真实数据或启动大规模搜索。